

PHSX 614 – Midterm Report

Interplanetary Trajectory Optimization Quantum Search

Sam Brauer

Department of Physics and Astronomy
University of Kansas
Lawrence, KS 66045, USA

PHSX 614 – Introduction to Quantum Algorithms

October 2026

Contents

1	Introduction	2
2	Underlying Physics	2
3	Quantum Algorithm: Dürr–Høyer Minimum Finding	3
3.1	Grover Search	3
3.2	Dürr–Høyer Iteration	4
4	Implementation	4
5	Algorithm	4
6	Preliminary Results	5
7	Future Work	8

1 Introduction

Interplanetary trajectory optimization can be broken down into three parts: the exit phase, the coast phase, and the capture phase. Finding the optimal trajectory is a difficult and time-consuming task, but breaking the problem into these components can make the process more manageable. Using the current orbital equations, portions of this process can be performed analytically rather than numerically. There are also other methods that can be used to accelerate the search, with different methods offering different benefits and drawbacks.

The overall physics in this project is performed classically, while the search for the optimal trajectory is performed using quantum algorithms. Specifically, the search for the minimum is performed using Grover’s algorithm, or more specifically, the Dürr–Høyer minimum-finding algorithm. Grover’s algorithm searches for a specific marked item in an unsorted set, while Dürr–Høyer extends this idea to search for the minimum-valued item in an unsorted set.

In this project, the set consists of weighted trajectory costs. The weights are chosen so that the optimization favors a trajectory in which the spacecraft is captured into Mars orbit while requiring minimal fuel, represented primarily by the required change in velocity, Δv .

2 Underlying Physics

As this project focuses primarily on quantum algorithms, the underlying orbital physics was implemented using the Python package *lamberthub*. Rather than finding a separate dataset on which to test the Dürr–Høyer algorithm, this approach maintains the original goal of orbital trajectory optimization while reducing the amount of additional code required to generate the physics data.

Given the central gravitational body, the initial position, the final position, and several additional parameters, Lambert’s problem can be used to determine the transfer velocities at the beginning and end of the trajectory. These velocities are then used to determine the change in velocity required for departure and capture. The resulting Δv provides a direct measure of the approximate fuel cost of the trajectory.

To create the search space, multiple initial positions, final positions, and mission durations were used to form a matrix of candidate trajectories. The grid spans one Earth–Mars synodic period for launch day, from 0 to 780 days, divided evenly into 32 points. The time of flight (TOF) spans a typical transfer window from 150 to 300 days, also divided into 32 points.

This produces a 32×32 grid containing

$$N = 32 \times 32 = 1024 = 2^{10} \tag{1}$$

candidate trajectories. Because the number of candidates is an exact power of two, each candidate can be represented by a unique state of 10 qubits, with no unused computational-basis states.

The final value assigned to each grid point is the weighted cost of the corresponding trajectory. The weighting is designed primarily to favor trajectories requiring lower fuel consumption and, consequently, lower total Δv .

3 Quantum Algorithm: Dürr–Høyer Minimum Finding

Grover’s algorithm [1] is a quantum algorithm used to search for a marked item in an unsorted list. It can find a marked item using $\mathcal{O}(\sqrt{N})$ oracle queries, compared with $\mathcal{O}(N)$ queries for a classical exhaustive search.

Dürr and Høyer [2] extended this approach to finding the minimum value in an unsorted list. Their algorithm has an expected query complexity of $\mathcal{O}(\sqrt{N})$. Instead of searching for a single predetermined item, the algorithm repeatedly performs a Grover search using a threshold based on the current best-known value. When a smaller value is found, the threshold is lowered and the search is repeated.

3.1 Grover Search

The Grover search begins by applying the Walsh–Hadamard transform to the qubits. This places the quantum register into an approximately uniform superposition of all candidate states.

The oracle then assigns different phases to candidate states depending on whether their costs satisfy the current threshold. In this implementation, the oracle is represented by a diagonal unitary operator. Candidate states with costs below the threshold receive a phase of -1 , while the remaining states receive a phase of $+1$.

The oracle is followed by the Grover diffusion operator, which amplifies the probability amplitudes associated with the marked states while reducing the amplitudes of the unmarked states.

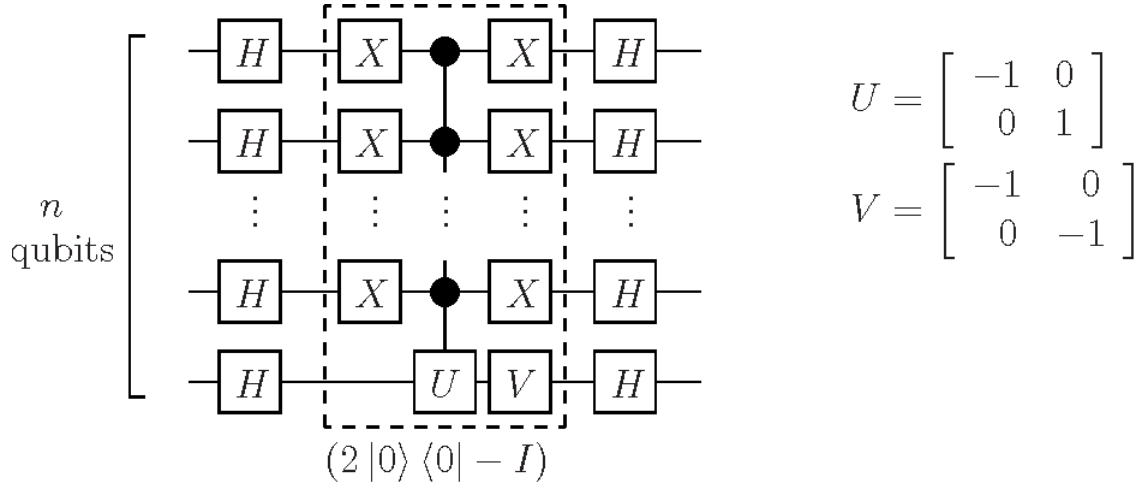


Figure 1: Quantum circuit for the Grover diffusion operator G over n qubits [6].

The diffusion operator can be written as

$$G = 2|s\rangle\langle s| - I = H^{\otimes n} (2|0\rangle\langle 0| - I) H^{\otimes n}, \quad (2)$$

where n is the number of qubits and $|s\rangle$ is the uniform superposition state,

$$|s\rangle = H^{\otimes n}|0\rangle = \frac{1}{\sqrt{2^n}} \sum_{i=0}^{2^n-1} |i\rangle. \quad (3)$$

The Grover diffusion circuit used in the project is shown in Figure 1.

3.2 Dürr–Høyer Iteration

Dürr–Høyer minimum finding is implemented as an iterative process. The algorithm begins with a randomly selected candidate as the current best value. A Grover search is then performed to identify a candidate with a lower cost. If a lower-cost candidate is found, it becomes the new best candidate and the threshold is updated.

This process is repeated until the probability of obtaining the minimum candidate becomes sufficiently high. Unlike a conventional Grover search, the marked state is therefore not fixed in advance; it changes as the current best solution improves.

4 Implementation

The quantum portion of the circuit currently runs using PennyLane’s `default.qubit` simulator with 10 qubits. The algorithm has not yet been executed on physical quantum hardware.

The most significant implementation change required for real hardware would involve the method used to determine probabilities. On the simulator, the `probs` function can directly calculate the probability associated with every computational-basis state. On real quantum hardware, this would instead require repeated measurements (shots) to estimate the probability distribution.

Each candidate trajectory is first evaluated using the classical orbital calculations. The resulting candidates are then assigned costs based primarily on the fuel requirement, represented by the required change in velocity. The Dürr–Høyer algorithm subsequently searches this collection of costs for the minimum.

5 Algorithm

The following code shows the quantum portion of the implementation. The Grover search is executed iteratively as part of the Dürr–Høyer minimum-finding procedure.

The implementation is based in part on the PennyLane `GroverOperator` implementation [7].

```

1 dev = qp.device("default.qubit", wires=wires)
2
3 @qp.qnode(dev)
4 def GroverSearch(num_iterations=1):
5     for wire in wires:
6         qp.Hadamard(wire)
7
8     for _ in range(num_iterations):
9         oracle()
10        qp.templates.GroverOperator(wires=wires)

```

```

11
12     return qp.probs(wires)
13
14 where oracle() is:
15     def oracle_diag(threshold_cost):
16         return np.where(costs_diag < threshold_cost, -1.0, 1.0).astype(
17             complex)
18
19     marks = oracle_diag(threshold_cost)
20
21     for _ in range(n_iterations):
22         qp.DiagonalQubitUnitary(marks, wires=wires)

```

Listing 1: PennyLane implementation of the Grover search and threshold oracle.

The key component of the oracle is the threshold comparison. For a given threshold cost, every candidate satisfying

$$C_i < C_{\text{threshold}} \quad (4)$$

is marked by assigning it a phase of -1 . The Grover diffusion operator then amplifies the probability of measuring these marked states.

6 Preliminary Results

The preliminary results are shown in Figures 2, 3, and 4.

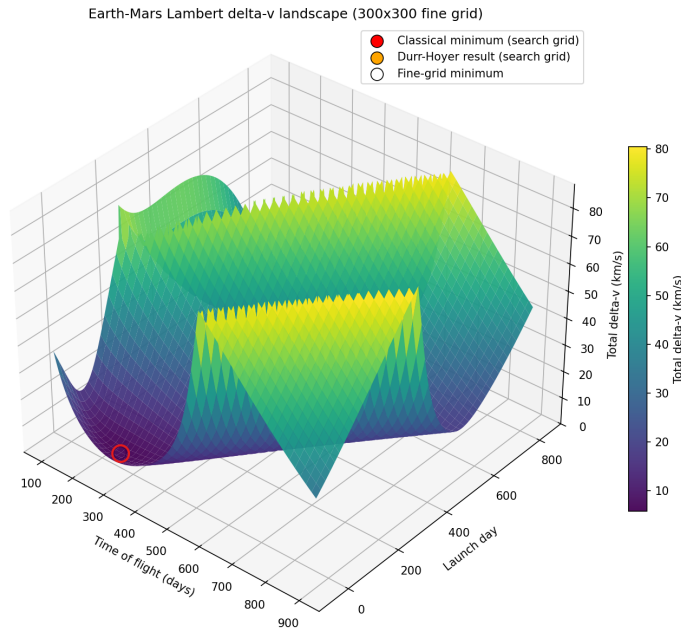


Figure 2: Preliminary trajectory optimization result.

Figure 2 shows the fuel consumption over a wide range of start times and mission lengths. The red circle shows where the optimal path lies for the least amount of fuel used.

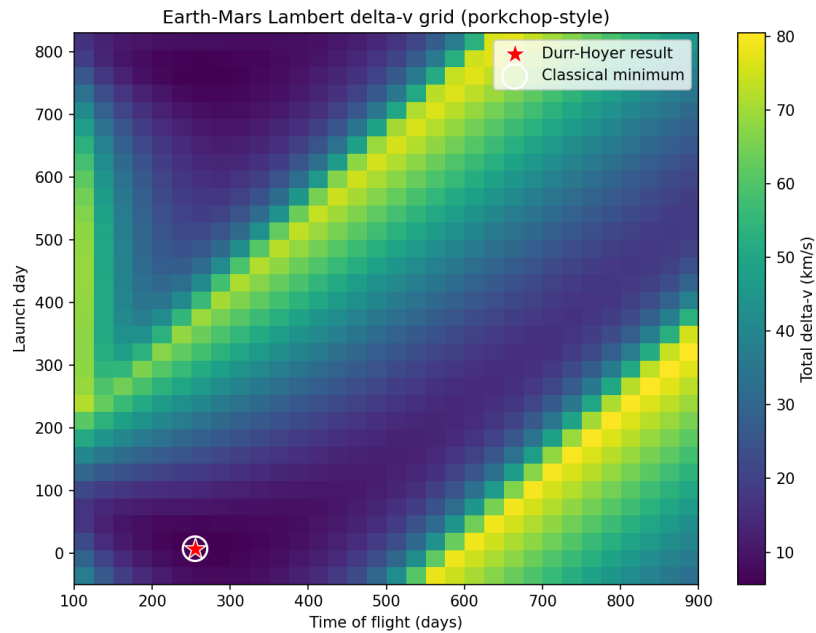


Figure 3: Preliminary cost landscape for the Earth–Mars trajectory search.

Figure 3 shows the same plot as figure 2 but a 2D representation from the top. It can be hard to tell however the plot repeats over the launch day axis as the synodic period of earth and mars is 780 days. to encompass the whole synodic period the launch days started 50 days before 0 and goes past 780 by 50 days. The common mission time of a mars mission is roughly 200-500 day, to fully encapsulate this a time of 100-900 days was chosen. I could not start at 0 or 1 as that would imply a very short mission and the velocity required to get from earth to mars in a day is an incredible value.

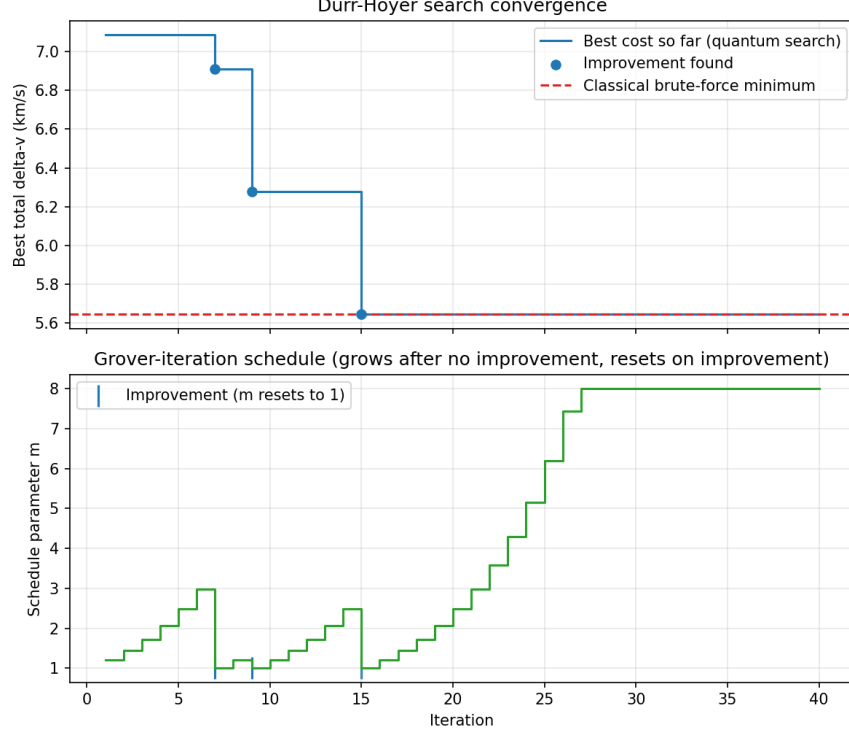


Figure 4: Preliminary results from the Dürr–Høyer minimum search.

Figure 4 shows the iterations and weights found by the Dürr–Høyer algorithm using Grover. The iterations show how the grover algorithm was ran 40 times inside the Dürr–Høyer. The top plot gives the lowest weight found in the search and then the bottom plot gives the scheduling. If a lower value is found it resets to 1 and if a value isnt found it multiplies the value by 1.2, this in essence is widening the search the longer it can’t find an answer.

Table 1: Comparison of classical brute-force search and the Dürr–Høyer quantum search.

Quantity	Classical brute force	Dürr–Høyer
Best total Δv (km/s)	5.642	5.642
Launch day / TOF (days)	6.8 / 254.8	6.8 / 254.8
Number of cost evaluations	1024 (full grid)	82
Correct minimum recovered?	—	Yes

The preliminary results demonstrate that the quantum minimum-finding procedure is capable of recovering the same minimum trajectory as the classical brute-force search. In the current 32×32 search space, the minimum total Δv is approximately 5.65 km/s, corresponding to a launch day of 257 days in the sampled grid.

The number of oracle calls is the primary quantity that still needs to be extracted from the implementation. This measurement is more important for the current midterm evaluation than wall-clock runtime. Because the current implementation uses a classical simulator, the simulation itself does not demonstrate a physical quantum speedup.

7 Future Work

The full project extends the preliminary Lambert-based calculation to real N-Body two-burn transfers. These calculations will use Runge–Kutta fourth-order (RK4) integration and be vectorized across candidate trajectories. The same Dürr–Høyer search loop will be used to identify low-cost trajectories.

The full search is staged into a multi-start launch search with many more parameters than the original two. As the final version will be using N-Body there will be variables such as initial velocity, launch time, launch angle in all axis, and then there will be the same variables for the second burn upon entering mars orbit.

The remaining work includes:

- integrating the quantum minimum-finding routine with the full N -body search, likely 3 dimensional;
- measuring qubit-count and circuit-depth scaling;
- comparing the Lambert-based optimum with the true N -body optimum;
- determining the total number of oracle evaluations required by the Dürr–Høyer search
- possibly executing a small version of the problem on real quantum hardware.
- looking into implementing quantum into the physics as well the search

These steps will allow the final project to evaluate not only whether the quantum algorithm can identify the correct trajectory, but also how the computational requirements scale as the trajectory search becomes more physically realistic. Although these improvements are more on the classical scale and the quantum portion is remaining the same, This test was done to see if the quantum worked. Now that we can see that the Dürr–Høyer algorithm works the full scale physics can be implemented. These improvements are to show more how even in real physics with different weights Dürr–Høyer can find a highly probably if not ideal answer.

References

- [1] L. K. Grover, “A fast quantum mechanical algorithm for database search,” *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [2] C. Dürr and P. Høyer, “A quantum algorithm for finding the minimum,” arXiv:quant-ph/9607014, 1996.
- [3] D. Izzo, “Revisiting Lambert’s problem,” *Celestial Mechanics and Dynamical Astronomy*, vol. 121, no. 1, 2015.
- [4] V. Bergholm et al., “PennyLane: Automatic differentiation of hybrid quantum-classical computations,” arXiv:1811.04968, 2018.
- [5] J. Piloto, *lamberthub: Lambert’s problem solvers in Python*. Available at: <https://github.com/jorgepiloto/lamberthub>.
- [6] “Strength and Weakness in Grover’s Quantum Search Algorithm,” *Semantic Scholar*. Available at: <https://www.semanticscholar.org/paper/Strength-and-Weakness-in-Grover’s-Quantum-Search-Younes/c157fffb9ed05bc24ec97dad785c293ca8aeaf2d>. Accessed October 6, 2026.
- [7] PennyLane, “GroverOperator.” Available at: <https://docs.pennylane.ai/en/stable/code/api/pennylane.GroverOperator.html>.